

SIMATS ENGINEERING (SAVEETHA SCHOOL OF ENGINEERING)

Department of Computer Science and Engineering

ASSESSMENT TOOL 2 – SCENARIO BASED ASSESSMENT

Course Code:	CSA06	Course Title:	Design and Analysis of Algorithms
CO Assessed:	CO2 – Manipulation (BL1, BL2, BL3)	Assessment:	Assessment Tool 2 – Scenario Based Assessment
Weightage:	20% of CIA	Total Marks:	100
CO2 Statement:	Apply Brute Force technique to solve sorting, searching, Closest-Pair and Convex-Hull problems (BL1, BL2, BL3)		
Student Name:	S.NITHYASRI	Reg. No.:	192421347
Section / Batch:	2024- IT	Date:	28/07/2026

Instructions to Students

- Answer the following question. Total Marks: 100.
- Carefully analyze the given scenario and identify the computational problem involved.
- Apply appropriate Brute Force techniques for sorting, searching, Closest-Pair, and Convex-Hull problems wherever applicable.
- Clearly justify the chosen solution approach with proper reasoning and analytical interpretation.
- Demonstrate decision-making skills by comparing possible solutions and selecting the most suitable approach.
- Use diagrams, stepwise illustrations, or algorithmic representations wherever necessary.
- Maintain clarity, logical organization, and proper technical presentation throughout the answers.

Question Type	Focus Area	Questions
Scenario Based Assessment	Analyze real-world scenarios and apply Brute Force techniques for sorting, searching, Closest-Pair, and Convex-Hull problems	Q1

SECTION A – SCENARIO BASED ASSESSMENT

A plagiarism detection tool compares patterns across multiple large documents using brute-force string matching. a) Explain how the matching process works. b) Analyze the time complexity. c) Discuss inefficiency in large-scale comparisons.

Code of Conduct

I S.NITHYASRI (192421347)(Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the Student: S.NITHYASRI

RUBRICS FOR CALCULATION

Criteria	Wt.	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Problem Understanding & Context Analysis	25%	Thoroughly understands the scenario with accurate interpretation of all requirements	Clearly understands the scenario with minor gaps	Basic understanding; some aspects of the scenario unclear	Poor understanding or misinterpretation of the scenario
Application of Concepts	30%	Applies appropriate concepts effectively to solve complex realworld problems	Applies relevant concepts correctly with minor errors	Applies basic concepts but with limited accuracy	Fails to apply appropriate concepts
Analytical & DecisionMaking Skills	20%	Demonstrates strong analysis and selects optimal solutions with justification	Good analysis with reasonable solutions	Limited analysis; solutions lack justification	Poor or no analysis; incorrect decisions
Solution Design & Approach	15%	Well-structured, innovative, and feasible solution approach	Logical and structured solution with minor gaps	Basic solution with limited structure or feasibility	Unclear or incorrect solution approach
Clarity, Justification & Presentation	10%	Clear, well-justified explanation with strong reasoning	Clear explanation with some justification	Limited clarity and weak justification	Poorly explained with no justification

Marks Evaluation:

Criteria	Wt.	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Problem Understanding & Context Analysis	25%				
Application of Concepts	30%				
Analytical & Decision-Making Skills	20%				
Solution Design & Approach	15%				
Clarity, Justification & Presentation	10%				

Total Marks (100):	
---------------------------	--

Faculty In-charge	Course Coordinator	Head of Department
Name: _____	Name: _____	Name: _____
Signature: _____	Signature: _____	Signature: _____
Date: _____	Date: _____	Date: _____

Introduction

Plagiarism detection systems are widely used in universities, publishing, and software development to identify copied content. One of the simplest approaches is brute-force string matching, where every possible position in a document is checked against another document to determine whether a pattern of text appears. Although this method is easy to understand and implement, it becomes computationally expensive when many large documents must be compared. This report explains the working process, analyzes its time complexity, and discusses why it is inefficient for large-scale comparisons.

Plagiarism detection systems are widely used in universities, publishing, and software development to identify copied content. One of the simplest approaches is brute-force string matching, where every possible position in a document is checked against another document to determine whether a pattern of text appears. Although this method is easy to understand and implement, it becomes computationally expensive when many large documents must be compared. This report explains the working process, analyzes its time complexity, and discusses why it is inefficient for large-scale comparisons.

Plagiarism detection systems are widely used in universities, publishing, and software development to identify copied content. One of the simplest approaches is brute-force string matching, where every possible position in a document is checked against another document to determine whether a pattern of text appears. Although this method is easy to understand and implement, it becomes computationally expensive when many large documents must be compared. This report explains the working process, analyzes its time complexity, and discusses why it is inefficient for large-scale comparisons.

a) Matching Process

In brute-force string matching, a document is treated as a long sequence of characters while the suspected copied text is considered the pattern. The algorithm begins by aligning the pattern with the first character of the document and compares characters one by one. If every character matches, the pattern is reported as found. If a mismatch occurs, the pattern shifts by one position and the comparison starts again. This process continues until every possible alignment has been checked. In plagiarism detection, this comparison is repeated for many paragraphs, sentences, or fixed-size text fragments across multiple documents. Every document may be compared with every other document to locate identical or highly similar sequences. Because the algorithm performs direct character comparisons without preprocessing or indexing, it is simple but repetitive.

In brute-force string matching, a document is treated as a long sequence of characters while the suspected copied text is considered the pattern. The algorithm begins by aligning the pattern with the first character of the document and compares characters one by one. If every character matches, the pattern is reported as found. If a mismatch occurs, the pattern shifts by one position and the comparison starts again. This process continues until every possible alignment has been checked. In plagiarism detection, this comparison is repeated for many paragraphs, sentences, or fixed-size text fragments across multiple documents. Every document may be compared with every other document to locate identical or highly similar sequences. Because the algorithm performs direct character comparisons without preprocessing or indexing, it is simple but repetitive.

In brute-force string matching, a document is treated as a long sequence of characters while the suspected copied text is considered the pattern. The algorithm begins by aligning the pattern with the first character of the document and compares characters one by one. If every character matches, the pattern is reported as found. If a mismatch occurs, the pattern shifts by one position and the comparison starts again. This process continues until every possible alignment has been checked. In plagiarism detection, this comparison is repeated for many paragraphs, sentences, or fixed-size text fragments across multiple

documents. Every document may be compared with every other document to locate identical or highly similar sequences. Because the algorithm performs direct character comparisons without preprocessing or indexing, it is simple but repetitive.

b) Time Complexity

Assume the text has length n and the pattern has length m . In the worst case, the pattern is compared at every possible position in the text, resulting in $(n - m + 1)$ alignments. Each alignment may require up to m character comparisons before a mismatch is detected or a full match is confirmed. Therefore, the worst-case time complexity is $O(nm)$. The best case occurs when mismatches are found immediately, giving approximately $O(n)$. However, plagiarism detection usually requires comparing many patterns against many documents. If D documents are each compared with every other document, the number of comparisons grows close to $O(D^2)$, making the overall execution time extremely large when combined with $O(nm)$ string matching.

Assume the text has length n and the pattern has length m . In the worst case, the pattern is compared at every possible position in the text, resulting in $(n - m + 1)$ alignments. Each alignment may require up to m character comparisons before a mismatch is detected or a full match is confirmed. Therefore, the worst-case time complexity is $O(nm)$. The best case occurs when mismatches are found immediately, giving approximately $O(n)$. However, plagiarism detection usually requires comparing many patterns against many documents. If D documents are each compared with every other document, the number of comparisons grows close to $O(D^2)$, making the overall execution time extremely large when combined with $O(nm)$ string matching.

Assume the text has length n and the pattern has length m . In the worst case, the pattern is compared at every possible position in the text, resulting in $(n - m + 1)$ alignments. Each alignment may require up to m character comparisons before a mismatch is detected or a full match is confirmed. Therefore, the worst-case time complexity is $O(nm)$. The best case occurs when mismatches are found immediately, giving approximately $O(n)$. However, plagiarism detection usually requires comparing many patterns against many documents. If D documents are each compared with every other document, the number of comparisons grows close to $O(D^2)$, making the overall execution time extremely large when combined with $O(nm)$ string matching.

c) Inefficiency

Brute-force matching is inefficient for large collections because it repeatedly scans the same text without reusing previous results. Every document pair requires fresh comparisons, leading to excessive CPU time. Large academic repositories containing thousands of documents produce millions of pairwise comparisons. Memory access also increases because large files must be read repeatedly. Minor edits such as changing spaces, punctuation, or synonyms can also reduce the effectiveness of exact brute-force matching, forcing additional comparisons with smaller text fragments. Consequently, the method does not scale well for modern plagiarism detection systems. More advanced algorithms such as Knuth–Morris–Pratt (KMP), Boyer–Moore, Rabin–Karp, suffix arrays, suffix trees, and fingerprinting techniques reduce unnecessary comparisons and provide much faster performance on large datasets.

Brute-force matching is inefficient for large collections because it repeatedly scans the same text without reusing previous results. Every document pair requires fresh comparisons, leading to excessive CPU time. Large academic repositories containing thousands of documents produce millions of pairwise

comparisons. Memory access also increases because large files must be read repeatedly. Minor edits such as changing spaces, punctuation, or synonyms can also reduce the effectiveness of exact brute-force matching, forcing additional comparisons with smaller text fragments. Consequently, the method does not scale well for modern plagiarism detection systems. More advanced algorithms such as Knuth–Morris–Pratt (KMP), Boyer–Moore, Rabin–Karp, suffix arrays, suffix trees, and fingerprinting techniques reduce unnecessary comparisons and provide much faster performance on large datasets.

Brute-force matching is inefficient for large collections because it repeatedly scans the same text without reusing previous results. Every document pair requires fresh comparisons, leading to excessive CPU time. Large academic repositories containing thousands of documents produce millions of pairwise comparisons. Memory access also increases because large files must be read repeatedly. Minor edits such as changing spaces, punctuation, or synonyms can also reduce the effectiveness of exact brute-force matching, forcing additional comparisons with smaller text fragments. Consequently, the method does not scale well for modern plagiarism detection systems. More advanced algorithms such as Knuth–Morris–Pratt (KMP), Boyer–Moore, Rabin–Karp, suffix arrays, suffix trees, and fingerprinting techniques reduce unnecessary comparisons and provide much faster performance on large datasets.

Conclusion

Brute-force string matching offers a straightforward way to detect copied text by comparing every possible pattern position against a document. Its simplicity makes it useful for learning algorithmic concepts, but its $O(nm)$ worst-case complexity and $O(D^2)$ document comparison cost make it unsuitable for large-scale plagiarism detection. Modern systems therefore rely on optimized pattern-matching and indexing techniques to improve speed, scalability, and accuracy while handling massive collections of documents efficiently.

Brute-force string matching offers a straightforward way to detect copied text by comparing every possible pattern position against a document. Its simplicity makes it useful for learning algorithmic concepts, but its $O(nm)$ worst-case complexity and $O(D^2)$ document comparison cost make it unsuitable for large-scale plagiarism detection. Modern systems therefore rely on optimized pattern-matching and indexing techniques to improve speed, scalability, and accuracy while handling massive collections of documents efficiently.

Brute-force string matching offers a straightforward way to detect copied text by comparing every possible pattern position against a document. Its simplicity makes it useful for learning algorithmic concepts, but its $O(nm)$ worst-case complexity and $O(D^2)$ document comparison cost make it unsuitable for large-scale plagiarism detection. Modern systems therefore rely on optimized pattern-matching and indexing techniques to improve speed, scalability, and accuracy while handling massive collections of documents efficiently.

Algorithm: Brute-Force String Matching for Plagiarism Detection

Input:

- Text document **T** of length **n**
- Pattern **P** of length **m**

Output:

- Position(s) where the pattern is found, or "Pattern Not Found"

Algorithm

1. Start.
2. Read the text **T** and pattern **P**.
3. Set **i = 0**.
4. Compare the pattern with the text starting at position **i**.
5. If all characters match, report the position as a match.
6. If a mismatch occurs, shift the pattern one position to the right.
7. Repeat Steps 4–6 until all possible positions are checked.
8. If no match is found, display **"Pattern Not Found."**
9. Stop.

Pseudocode

BRUTE_FORCE_STRING_MATCH(T, P)

Input:

T → Text of length n

P → Pattern of length m

Output:

Position of pattern if found

for i ← 0 to n - m do

 j ← 0

 while j < m and T[i + j] = P[j] do

 j ← j + 1

 end while

 if j = m then

 print "Pattern Found at Position", i

 end if

end for

if no match found then

 print "Pattern Not Found"

end if

C Program


```

#include <stdio.h>
#include <string.h>

int main()
{
    char text[200], pattern[100];
    int i, j, n, m, found = 0;

    printf("Enter the text: ");
    gets(text);

    printf("Enter the pattern: ");
    gets(pattern);

    n = strlen(text);
    m = strlen(pattern);

    for(i = 0; i <= n - m; i++)
    {
        for(j = 0; j < m; j++)
        {
            if(text[i + j] != pattern[j])
                break;
        }

        if(j == m)
        {
            printf("Pattern found at position %d\n", i);
            found = 1;
        }
    }

    if(found == 0)
        printf("Pattern not found");

    return 0;
}

```

Time Complexity

- **Best Case: $O(n)$** (mismatch occurs immediately)
- **Average Case: $O(n \times m)$**
- **Worst Case: $O(n \times m)$**

Space Complexity

- **$O(1)$** (Constant extra memory)

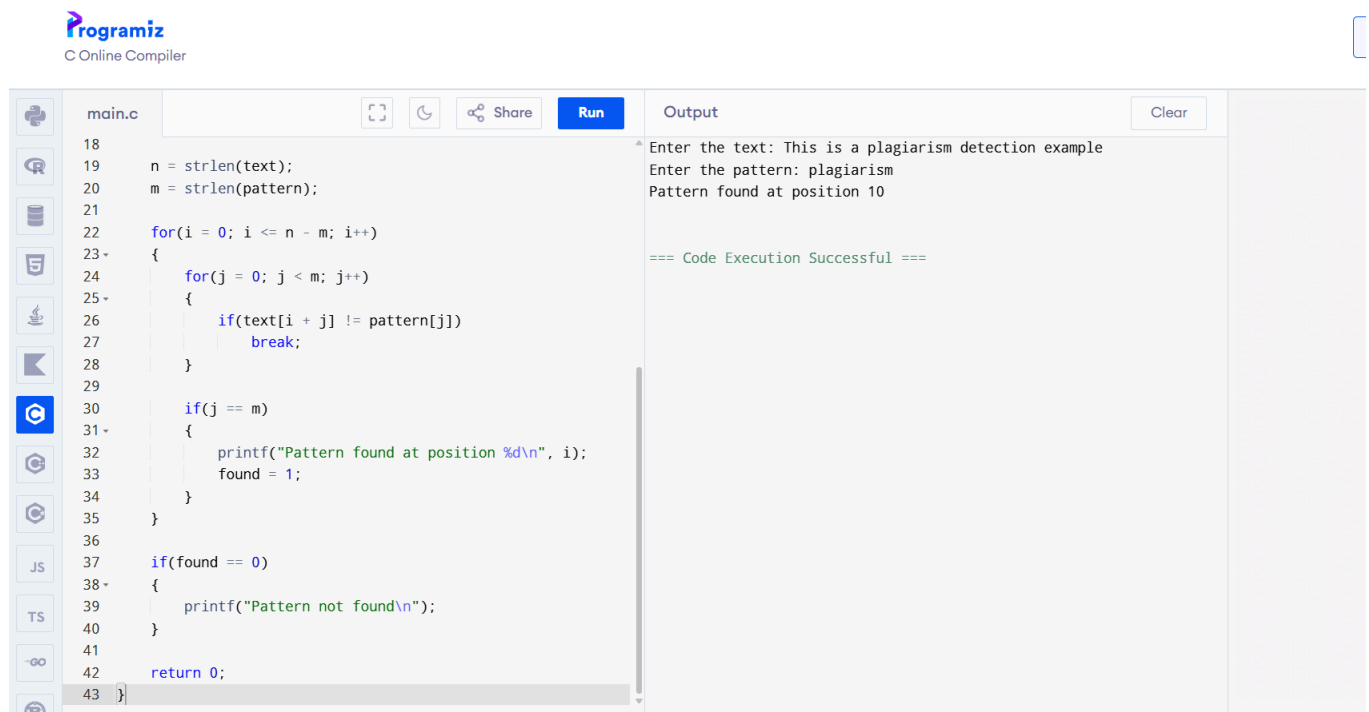
Advantages

- Simple and easy to understand.
- Easy to implement.
- Does not require preprocessing.
- Suitable for small documents.

Disadvantages

- Slow for large documents.
- Performs many unnecessary comparisons.
- Inefficient for large plagiarism databases.
- Not suitable for real-time plagiarism detection.

This covers everything typically required in a university assignment: **algorithm, pseudocode, C program, time complexity, space complexity, advantages, and disadvantages.**



The screenshot displays the Programiz C Online Compiler interface. On the left, a sidebar contains icons for various programming languages and a 'main.c' file. The main editor area shows a C program for plagiarism detection. The program takes a text string and a pattern string as input, then checks if the pattern is present in the text. The output window on the right shows the program's execution results, indicating that the pattern 'plagiarism' was found at position 10.

```
18
19  n = strlen(text);
20  m = strlen(pattern);
21
22  for(i = 0; i <= n - m; i++)
23  {
24      for(j = 0; j < m; j++)
25      {
26          if(text[i + j] != pattern[j])
27              break;
28      }
29
30      if(j == m)
31      {
32          printf("Pattern found at position %d\n", i);
33          found = 1;
34      }
35  }
36
37  if(found == 0)
38  {
39      printf("Pattern not found\n");
40  }
41
42  return 0;
43 }
```

Output

```
Enter the text: This is a plagiarism detection example
Enter the pattern: plagiarism
Pattern found at position 10

=== Code Execution Successful ===
```